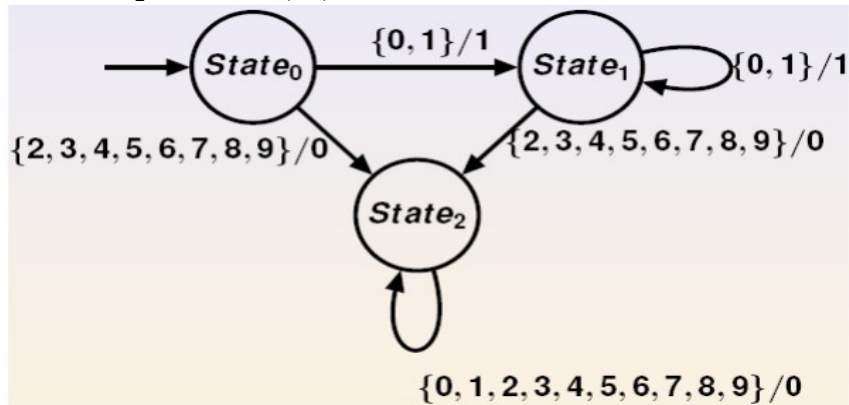


Grafički prikaz – 1 (74)

ovo je prvi transducer proizvodi nesto ne prima samo ulaz i da li prihata rec nego da i autput npr input „102 output je 110, prepoznavač u drugu ruku nema / i samo gleda da li rec pripada jeziku ili ne Umesto „opširnog“ opisa koristi se grafička notacija. To se naziva usmereni „graf“.

- Čvorovi su označeni imenima „stanja“: State0, State1, State2.
- Postoji (neoznačena) ivica koja označava „početnu tačku“.
- Ostale ivice imaju oznaku oblika {In}/Out.
- {In} je podskup mogućih ulaznih simbola.
- Out je jedan izlazni simbol.

U celini, ivica iz Statei u Statej sa oznakom {In}/Out može se tumačiti ovako:

Ako je „program“ stigao u stanje Statei i sledeći ulazni simbol pripada skupu In, tada se na izlazu daje simbol Out i prelazi se u stanje Statej.

Konačni automat – 0 (76)

Prethodni konačni automat (FSM) naziva se konačni **prepoznavač stanja**, dok se prvi FSM naziva **transduktor**.

Prepoznavač ima potpuno istu strukturu kao i transduktor.

Razlika je u tome što se, umesto da se **izlaz eksplicitno označava** kao deo oznake ivice, on podrazumeva razlikovanjem „posebnih“ stanja. Naime:

Stanja u kojima bi se, pri ulasku u njih, na izlazu dobila vrednost 1.

State1 je jedno takvo stanje.

Kao jedan odgovor na pitanje „šta se dešava u crnoj kutiji“, FSM-ovi nude bogat skup ideja i primena.

Među tim primenama su:

- Leksička analiza u kompajlerima: određivanje identifikatora i numeričkih konstanti prilikom analize programskog izraza.
- Opisivanje hardverskih sistema.

U ovom drugom kontekstu, svi hardverski sistemi se, na kraju, mogu opisati kao kompozicije konačnih automata-transduktora sa izlaznim simbolima {0, 1}.

Dizajn konačnog automata je standardni pristup pri razvoju digitalnog sistema.

Neformalni opisi koristili su pojmove „ulaznog“ i „izlaznog“ simbola.

„Crna kutija“ se posmatra u smislu „mapiranja“ sa „nizova“ ulaznih simbola na („nizove“) izlaznih simbola.

Ovi opisi su prilično neodređeni i netačni; npr. ulaz u primeru je {00, 01, 10, 11}.

Sada ćemo predstaviti **formalni okvir** unutar kojeg će biti postavljene naredne ideje.

Abeceda (81) Alfabet

Abeceda je (**konačan**) skup simbola.

$\text{Alphabet} = \{a_1, a_2, \dots, a_k\}$

- **Alphabet** označava proizvoljnu abecedu;
- **a** označava proizvoljan simbol iz Alphabet.

Primeri:

- Abeceda Decimal (decimalne cifre):

$\text{Decimal} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- Abeceda Binary (binarni brojevi):

$\text{Binary} = \{0, 1\}$

- Abeceda Roman (rimski slova):

$\text{Roman} = \{a, b, c, \dots, z, A, B, C, \dots, Z\}$

Reči nad abecedama (82)

Reč, w , nad abecedom A , je konačan niz simbola iz A

Napomene:

- Reč je **niz**, a ne skup.
- **Redosled** simbola je važan.

Primeri:

- Roman reči: dog, god, Java, word, seQuenCe
- Decimalne reči: 3, 10, 112289
- Binarne reči: 0, 1, 10, 10

Dužina reči (83)

Dužina reči w je broj simbola u w . Ovo ćemo označavati sa $|w|$.

Reč koja ima dužinu 0 naziva se **prazna reč**. Ona se uvek označava sa ϵ (epsilon).

Primeri:

- Decimalne reči: $|3|=1, |10|=2, |112289|=6$
- Binarne reči: $|0|=1, |1|=1, |10|=2$
- Rimske reči: $|\text{word}|=4, |\text{seQuenCe}|=8, |a|=1$

Spajanje reči (84)

Neka su u i v reči nad abecedom A .

Reč w koja nastaje spajanjem u i v je reč čiji je niz simbola uv

Dužina reči w je $|w| = |u| + |v|$

Ako je u ili v prazna reč ϵ , tada važi:

$u\epsilon = u, \epsilon v = v, \epsilon\epsilon = \epsilon$

k -struko spajanje reči www označava se kao:

$w^k = w \cdot w \cdot w \dots w$ (for $k \geq 0$)

Ponekad ćemo koristiti oznaku u^v da označimo spajanje reči u i v .

Examples Decimal: $3^{10} = 310, 22^{33^{\epsilon}} = 2233$

Binary: $0^1 = 01, 0^{\epsilon^0} = 00$

Roman: $w^o r^d = \text{word}$

Jezici reči – 0 (85)

Pojam **jezika** ima veliko značenje u računarstvu.

Za razliku od drugih pojmova uvedenih gore, jezici koji nas zanimaju su obično **beskonačni objekti**.

Jezik L nad abecedom A je **podskup skupa svih mogućih reči nad A** .

Praktično je koristiti **skraćenu oznaku** za „skup svih mogućih reči nad A “.

Pošto razmatramo samo reči **konačne dužine**, ovaj skup obuhvata:

0: sve reči nad A dužine 0, tj. ε (A^0),

1: sve reči nad A dužine 1 (A^1),

2: sve reči nad A dužine 2 (A^2),

...

k : sve reči nad A dužine k (A^k).

Oznaka A^k se koristi za **sve reči nad A dužine k** .

Dakle, „skup svih mogućih reči nad A “ je:

$$\bigcup_{k=0}^{\infty} A^k$$

i upravo se za njega koristi skraćenica A^*

Primeri:

- $\{w \in \text{Decimal}^* : |w| = 2\}$
- $\{w \in \text{Binary}^* : |w| > 3\}$ – sve binarne reči duže od 3
- $\{w \in \text{Roman}^* : |w| < 2\}$ – sve rimske reči kraće od 2

Operacije nad jezicima

Operacije nad jezicima (88)

Neka su L_1 i L_2 dva jezika. Definišu se sledeće operacije:

1. Unija:

$$L_1 \cup L_2 = \{w \mid w \in L_1 \text{ ili } w \in L_2\}$$

Sve reči koje pripadaju barem jednom od jezika.

2. Konkatencija (spajanje):

$$L_1 \cdot L_2 = \{w \mid w = w_1 w_2 \text{ za neko } w_1 \in L_1 \text{ i } w_2 \in L_2\}$$

Reč iz rezultujućeg jezika nastaje spajanjem reči iz L_1 i reči iz L_2 .

3. Kleeneova zvezda:

$L_1^* = \{w : w = w_1 w_2 \dots w_k \text{ for some } k \geq 0 \text{ and some } w_1, \dots, w_k \in L_1\}$

Sve reči koje se dobijaju spajanjem nula ili više reči iz L_1 .

Primeri:

Neka je $L_1 = \{01, 1, 00\}$ i $L_2 = \{11\}$, tada:

- **Unija:**

$L_1 \cup L_2 = \{01, 1, 00, 11\}$

- **Konkatenacija:**

$L_1 \cdot L_2 = \{0111, 111, 0011\}$

- **Kleeneova zvezda:**

$L_2^* = \{\epsilon, 11, 1111, 111111, \dots\}$

Regularni izraz – 0 (89)

Regularni izrazi nad abecedom A su reči nad abecedom

$A \cup \{(\,,\,),\,\emptyset,\, +,\, *\}$

ako i samo ako važe sledeća pravila:

1. **Osnovni regularni izrazi:**

$\emptyset$ i svaki simbol iz A je regularni izraz.

2. **Složeni regularni izrazi:**

Ako su R i S regularni izrazi, tada su i:

- (RS) – konkatenacija
- $(R+S)$ – unija
- R^* – Kleeneova zvezda

Primeri:

Neka je $A = \{0, 1\}$. Sledeći izrazi su regularni:

$1^*, 0^*, (1+0^*), 10^*, 0^*+\emptyset, \emptyset^*$

Međutim, sledeći **nisu regularni izrazi:**

$+, \cdot, \emptyset, (01^*$

Svaki regularni izraz odgovara jednom jeziku.

Za svaki regularni izraz R , jezik koji odgovara izrazu R , označen sa $L(R)$, definiše se kao:

1 $L(\emptyset) = \emptyset, L(a) = \{a\}$

• 2 if R and S are two regular expressions then $L((RS)) = L(R)L(S)$

$L((R+S)) = L(R) \cup L(S)$

$L(R^*) = L(R)^*$

Povratak na gradivo (91)

- Na ulaz simbola aaa prelazimo iz **State0** u **State1**, tj.
- **Abeceda** je konačan skup simbola.
- **Reči** nastaju spajanjem (konkatenacijom) simbola.
- **Jezik** je podskup skupa svih mogućih reči nad nekom abecedom.
- **Regularni izraz** je notacija koja služi za definisanje jezika.

Konačni automat – 0 (92)

Sada razmatramo „najjednostavniju“ klasu modela mašina, tj. **konačni automat (prepoznavać)**.

Formalno, konačni automat (FSM) se karakteriše sledećim elementima:

- **Stanja**
- **Ulazi**
- **Prelazi**
- **Početno stanje**
- **Prihvatljiva stanja**

Kažemo da je FSM definisan kao **5-torka**:

(States, Inputs, Transition, initial, Accept)

gde su:

- **States** – konačan skup stanja,
- **Inputs** – ulazna abeceda,
- **initial** $\in$ States – početno stanje,
- **Accept** $\subseteq$ States – skup prihvatljivih stanja,
- **Transition**: States $\times$ Inputs $\rightarrow$ States – funkcija prelaza.

Funkcija prelaza daje, za svako stanje qqq i svaki ulazni simbol aaa, novo stanje

Transition(q,a)\text{Transition}(q, a)Transition(q,a) koje nastaje kao reakcija na izvršenje simbola aaa u stanju qqq

Konačni automat – 3 (95)

Nije odmah očigledno, samo posmatranjem, koje **ulazne nizove** ovaj veći primer prepoznaje.

Koristićemo ovaj primer da ilustrujemo **opšti koncept jezika koji prepoznaje konačni automat**.

Razmotrimo FSM:

M = (States, Inputs, Transition, initial, Accept).

Primitite da za svako stanje $q \in$ States, Transition mapira svaki ulazni simbol $a \in$ Inputs u **tačno jedno sledeće stanje** q' , tj. Transition je **totalna i deterministička**.

Dakle, za svaku ulaznu reč $w \in \text{Inputs}^*$, „obrada“ reči (w) od strane (M) može se opisati kao **niz stanja koja se posećuju** dok se svaki simbol iz (w) „čita“.

Da li je (w) prihvaćena ili odbijena zavisi od toga da li je **poslednje stanje koje je automat posetio prihvatljivo**, tj. da li pripada skupu **Accept**.

Konačni automat – 6 (98)

Da bismo formalno opisali pojam „**niza posećenih stanja**“, generalizujemo funkciju prelaza Transition sa pojedinačnih simbola iz Inputs

na **reči iz Inputs^***

Za FSM:

$M = (\text{States}, \text{Inputs}, \text{Transition}, \text{initial}, \text{Accept})$

definišemo proširenu funkciju prelaza:

$\text{Transition}^*: \text{States} \times \text{Inputs}^* \rightarrow \text{States}$

na sledeći način:

$$\text{Transition}^*(q, w) = \begin{cases} q & \text{if } w = \epsilon \\ \text{Transition}^*(\text{Transition}^*(q, u), a) & \text{if } w = u \cdot a \end{cases}$$

Ovo znači:

- Ako je ulaz **prazna reč** ϵ , stanje ostaje q .
- Ako je ulaz reč $w = u \cdot a$, prvo se obradi deo u , zatim se primeni funkcija Transition na poslednje stanje i simbol a .

Konačni automat – 7 (99)

Primetite da je ova definicija **rekurzivna**:

- Slučaj $w = \epsilon$ označava: „**stanje ostaje q** “.
- Opšti slučaj ($w \neq \epsilon$) označava:
„**stanje u koje M dolazi iz q na reč $w = u \cdot a$ je stanje koje se dobije primenom funkcije prelaza Transition na stanje koje M dostigne na reči u i simbolu $a \in \{\text{Inputs}\}$** “.

Jezik $L(M)$ koji prepoznaje FSM

$M = (\text{States}, \text{Inputs}, \text{Transition}, \text{initial}, \text{Accept})$

Naravno! Evo šta svaki element u $M = (\text{States}, \text{Inputs}, \text{Transition}, \text{initial}, \text{Accept})$

- States** – skup svih **stanja** automata (konačan skup).
- Inputs** – **ulazna abeceda**, tj. skup simbola koje automat može da primi.
- Transition** – funkcija **prelaza**, koja svakom paru (stanje, ulazni simbol) dodeljuje novo stanje.
- initial** – **početno stanje**, stanje u kome automat počinje obradu ulaza.

5. **Accept** – skup **prihvatljivih (finalnih) stanja**; ako automat završi u nekom od ovih stanja, ulazna reč je prihvaćena.

sada se može definisati kao:

$$L(M) = \{w \in \text{Inputs} \mid \text{Transition}^*(\text{initial}, w) \in \text{Accept}\}$$

Primitite da je ova definicija **zasnovana na računanju**, tj. **nije „ad hoc“ opis**.

Ponašanja – 0 (102)

- Kao što smo rekli, FSM modelira **crnu kutiju sa ulazom**. Za svaki dati ulaz postoji jedan **tok izvršavanja (run)**. Taj tok je u stvari **test mašine na datom ulazu**.
- Takav tok opisuje **ponašanje mašine na tom ulazu**.
- **Ponašanje** se stoga definiše kao **niz stanja** kroz koja mašina prolazi.
- Na primer, mašina koja je u prethodnom primeru prihvatila reč ($w = 110$) prolazi kroz stanja (q_0), zatim (q_1), zatim (q_4) i završava u (q_5).
- Niz ($q_0 q_1 q_4 q_5$) predstavlja **ponašanje te mašine**.
- Pošto je (w) prihvaćena, ovo ponašanje nazivamo **ponašanjem prihvatanja (acceptance behaviour)**.

Nedeterministički konačni automat – 0 (104)

Ranije je napomenuto da je funkcija prelaza

$$\text{Transition} : \text{States} \times \text{Inputs} \rightarrow \text{States}$$

definisana tako da bude **totalna i deterministička**,

tj. da svaki par

(stanje, ulazni simbol) ima tačno **jedno** sledeće stanje.

Sada razmatramo efekat promene ove definicije tako da se dozvoli

nedeterminističnost u funkciji prelaza **Transition**.

To možemo uraditi na dva načina:

- dozvoliti **više od jednog izbora** sledećeg stanja, tj.
($\text{Transition}(q, a)$) ne mora biti jedinstveno;
- dozvoliti „prazne“ (ϵ) **prelaze** između stanja, tj. trenutno stanje se može promeniti **bez ikakvog ulaza**.

Nedeterministički konačni automat – 2 (106)

Formalno, nedeterministički konačni automat (NFSM), (M), opisuje se kao:

M = (States, Inputs, Transition, initial, Accept)

gde su **States**, **Inputs**, **initial** i **Accept** definisani isto kao ranije, ali je funkcija **Transition** sada preslikavanje:

Transition : States $\times$ Inputs $\rightarrow$ P(States)

pri čemu P(States)) označava **partitivni skup**, tj. **skup svih podskupova skupa States**.

Nedeterministički konačni automat – 5 (109)

U **determinističkom** automatu, polazeći od početnog stanja, svaka reč w e Inputs*, se posmatrati kao prolazak kroz **jedinstven niz (putanju) stanja**.

U **nedeterminističkom** automatu taj niz **nije nužno jedinstven**: za jednu istu reč (w) može postojati **više mogućih nizova stanja**.

Odnosno, jednoj reči (w) može odgovarati **više od jednog ponašanja** automata.

Nedeterministički konačni automat – 7 (111)

Definicija prihvatanja reči w e Inputs* od strane NFSM-a (M) može se, neformalno, iskazati na sledeći način:

„Reč (w) je prihvaćena od strane nedeterminističkog konačnog automata (M) ako postoji **bar jedna putanja izvršavanja** automata (M) nad rečju (w) koja se završava u nekom stanju (q e {Accept}).“

Dakle, u datom primeru, reč **010** je prihvaćena.

Treba obratiti pažnju na značaj toga da je **izvršavanje završeno**, tj. da **svi simboli reči w moraju biti pročitani**.

Nedeterministički konačni automat – 9 (113)

Promena sa **States** na P(States) kao kodomena funkcije **Transition** ima jasne posledice po proširenu funkciju prelaza **Transition***, a samim tim i po način na koji se definiše pojam

„**jezik L(M) koji prepoznaje NFSM M**“.

Nedeterministički konačni automat – 10 (114)

Sada možemo definisati analog funkcije **Transition*** za **nedeterminističke** mašine.

Za NFSM

$M = (\text{States}, \text{Inputs}, \text{Transition}, \text{initial}, \text{Accept})$ the definiše se **funkcija dostižnosti**:

$\text{Transition}^* : \text{States} \times \text{Inputs}^* \rightarrow \mathcal{P}(\text{States})$

pri čemu važi da je:

$is, \text{Transition}^*(q, w) =$

$is, \text{Transition}^*(q, w) =$

$\{q\}$ if $w = \varepsilon$

$Uq' \in \text{Transition}^*(q, u) \text{ Transition}(q', a) \text{ if } w = u^{\wedge}a$

Drugim rečima, za dati početni stanje (q) i ulaznu reč (w),

$(\text{Transition}^*(q, w))$ daje **skup svih stanja** do kojih je moguće doći čitanjem reči (w) iz stanja (q).

A sada imamo da je **jezik $L(M)$** koji prepoznaje

nedeterministički konačni automat (NFSM) M definisan kao:

$L(M) = \{w \in \text{Inputs}^* : \text{Transition}^*(\text{initial}, w) \cap \text{Accept} \neq \emptyset\}$

NFSM naspram FSM – 0 (117)

Pošto je funkcija prelaza **Transition** kod determinističkih automata (FSM) ograničen oblik funkcije prelaza koja je dozvoljena kod nedeterminističkih automata (NFSM), sledi da važi:

$\text{Jezici}(\text{FSM}) \subseteq \text{Jezici}(\text{NFSM})$

Postavlja se pitanje:

Da li NFSM može da prepoznaje „više jezika“ nego FSM?

Ispostavlja se da **ne dobijamo ništa novo u pogledu jezika** –

ne postoje jezici koje NFSM može da prepozna, a koje FSM ne može.

NFSM naspram FSM – 1 (118)

Teorema 1

Za svaki **nedeterministički konačni automat (NFSM)**

$M = (\text{States}, \text{Inputs}, \text{Transition}, \text{initial}, \text{Accept})$

postoji **deterministički konačni automat (FSM)**

$M_0 = (\text{States}', \text{Inputs}, \text{Transition}', \text{initial}', \text{Accept}')$

takav da važi:

$L(M) = L(M_0)$

Odnosno, za svaki NFSM možemo konstruisati odgovarajući FSM koji prepoznaje **isti jezik**.

Za dokaz pogledati i precrtati 92 slajd

ϵ -prelazi u NFSM – 1 (123)

Teorema 2

Za svaki **NFSM sa ϵ -prelazima**

$M = (\text{States}, \text{Inputs}, \text{Transition}, \text{initial}, \text{Accept})$

postoji **NFSM bez ϵ -prelaza**

$M_0 = (\text{States}', \text{Inputs}, \text{Transition}', \text{initial}', \text{Accept}')$

takav da važi:

$$L(M) = L(M_0)$$

Skica dokaza: vidi dodatak (appendix).

Kombinovanje FSM: Unija (124)

ϵ -prelazi omogućavaju da se automati „spoje“, što daje elegantan način za **kombinovanje jezika**.

Neka su M_1 i M_2 ϵ -NFSM-ovi koji prepoznaju jezike $L(M_1)$ i $L(M_2)$ (podskupove Inputs*) respektivno.

Tada spojeni automat prepoznaje $L(M_1) \cup L(M_2)$, tj. „ili“: $L(M_1) + L(M_2)$.

Kombinovanje FSM: Konkatenacija (125)

Spojeni automat prepoznaje $L(M_1) \cdot L(M_2)$, tj. **konkatenaciju jezika**.

Kombinovanje FSM: Iteracija (126)

Spojeni automat prepoznaje $(L(M_1))^*$, tj. **Kleeneovu zvezdu**, odnosno **0 ili više ponavljanja jezika $L(M_1)$** .

Verifikacija (127)

- Sada prelazimo na pitanje kako osigurati da **FSM zadovoljava originalne zahteve**.
- Za to uvodimo pojam **regularnih izraza**.
- Ovi regularni izrazi koriste se za **izražavanje zahteva** koje FSM treba da ispuni.
- Zatim ćemo koristiti **pravila konstrukcije** da **transformišemo regularni izraz u FSM**.

Regularni izrazi → FSM (128)

Neka je A abeceda:

1. $\emptyset$: Ovaj izraz se prevodi u mašinu koja **ne radi ništa**.
2. a , gde je $a \in A$: Ovaj izraz se prevodi u mašinu koja **završava kada primi simbol a** .
3. Ako su R i S dva regularna izraza,
 $(R + S)$ se prevodi u FSM pomoću **unije** jezika R i S .
4. (RS) se prevodi u FSM pomoću **konkatenacije** jezika R i S .
5. R^* se prevodi u FSM pomoću **Kleeneove zvezde**, tj. **0 ili više ponavljanja** jezika R .

Nedostaci FSM-ova (130)

- Dijagrami stanja i prelaza su „**ravni**“, tj. nemaju prirodnu hijerarhiju, dubinu ili modularnost.
- Dijagrami stanja i prelaza se vrlo lako mogu **zagušiti prelazima**, na primer kod složenih višerazinskih prekida (interrupts).
- Nisu **ekonomični u pogledu paralelne kompozicije**: broj stanja raste eksponencijalno kada se automati kombinuju paralelno.

Eksplוזija stanja (132)

Pretpostavimo da imamo dva konačna automata.

Konačni automat koji odgovara **paralelnoj kompoziciji** ova dva automata jasno pokazuje razlog **eksponencijalnog rasta broja stanja**:

- Ako svaki od automata ima po 100 stanja,
- tada spojeni automat će sadržati **100×100 kombinacija stanja**.

